# The P2P network simulation

`network_sim.py` puts the coding schemes into a simulated peer-to-peer network and measures what happens to them there. It is the Phase 3 half of the project: where `benchmark_1D_2D_BC.py` asks *what can this code survive*, this asks *can a node actually get the data and rebuild the blob before the slot ends*.

Those are genuinely different questions, and the second one cannot be answered by the first. A decode time of 40 seconds is a fact about arithmetic. Whether 40 seconds is acceptable is a fact about a system with a 12-second slot, a deadline, a finite link and peers that may not answer.

## What the project brief asks for, and where it now stands

Phase 3 of the proposal specifies prototypes "deployed within a controlled simulated peer-to-peer (P2P) network environment", evaluated "across standard network architectures", capturing "bandwidth saturation, packet propagation latency, and block processing throughput" under "varying adversarial blockchain network conditions".

| Requirement | Status |
|---|---|
| Simulated P2P environment | done — SimPy discrete-event, 64 peers, custody assignment |
| Packet propagation latency | done — per-peer lognormal latency, reported as gather time |
| Bandwidth saturation | done — one contended downlink; saturates under multi-blob load |
| Block processing throughput | done — blobs reconstructed per slot |
| Varying adversarial conditions | done — swept 0–70% withholding peers |
| Randomly generated blob data | done — real EIP-4844 packing, shared blob |
| Data reconstruction success rate | done — per-trial, byte-verified |
| Scalability in node count | done — peer count is a swept dimension |
| **Standard network architectures (plural)** | **not done — one topology only** |

The remaining gap is listed again at the end rather than buried. Everything above it is measured.

## How SimPy works, in the three concepts this uses

SimPy is a discrete-event simulator. Nothing in it runs in real time.

**The environment is a clock and an event queue.** `env.now` is the current *simulated* time in seconds. `env.run(until=4.0)` processes every scheduled event up to simulated t=4s and stops.

**A process is an ordinary Python generator.** `yield env.timeout(0.08)` means "resume me 80 simulated milliseconds from now". The scheduler jumps straight to the next event rather than waiting, so simulating a 12-second slot across 64 peers costs microseconds of real CPU. This is what makes sweeping hundreds of network configurations cheap.

**A resource models contention.** The node's downlink is a `simpy.Resource` of capacity 1: a transfer acquires it, holds it for `bytes / bandwidth` seconds, releases it. Because every transfer queues for the same object, bandwidth saturation *emerges* from demand rather than being asserted — ask for more than the link can carry before the deadline and the queue backs up until the stragglers miss it.

## The one idea the design rests on

In the coding benchmark, the erasure mask is an **input**: `random_mask(rate, rng)` invents one at a nominal rate. Here it is an **output** — whatever the node failed to collect in time.

```
benchmark:    mask = adapter.random_mask(rate, rng)     # invented
network sim:  mask = <what did not arrive before the deadline>
```

Everything downstream of the mask — decode, extraction, byte-level verification — is the *same adapter code* the benchmark already uses, untouched. That single substitution is the entire integration surface, which is why this file contains no coding logic of its own. A fix to `2D_RS_Galois.py` changes these results automatically.

This buys something beyond convenience. The benchmark drops symbols independently; real failures are not independent, because when a peer goes offline *everything it custodied* vanishes at once. A peer-failure model produces those **clustered** erasures for free, from the topology, rather than having to inject correlation by hand — and the benchmark report explicitly flags i.i.d.-only testing as a limitation.

## What is modelled

**Peers and custody.** 64 peers. Each cell is held by `custody_replication` of them (default 4), so a cell is lost only if every custodian fails. This is the *network's* redundancy, and it is separate from the code's — an important distinction, because it turns out to absorb most peer failure before the erasure code is asked to do anything.

**Cells.** The unit a node requests is a PeerDAS-shaped cell: 64 field elements × 32 bytes, plus a 48-byte KZG proof, so 2096 bytes.

**Adversaries.** A fraction of peers never answer. The client burns its request timeout before falling back to another custodian, so withholding costs the victim *time* as well as data — which is the point of modelling it rather than just deleting symbols.

**The deadline.** A 4-second sampling window inside a 12-second slot. Anything not received by then is an erasure, full stop.

### How much a node actually has to download

This is the part worth dwelling on, because getting it wrong produced nonsense on the first attempt.

The obvious answer is $k$ symbols — the information-theoretic floor. That is right only for an MDS code decoded optimally. In general a node must fetch enough that what remains *missing* falls inside its own decoder's tolerance:

> symbols needed $= n \times (1 - \text{tolerance})$

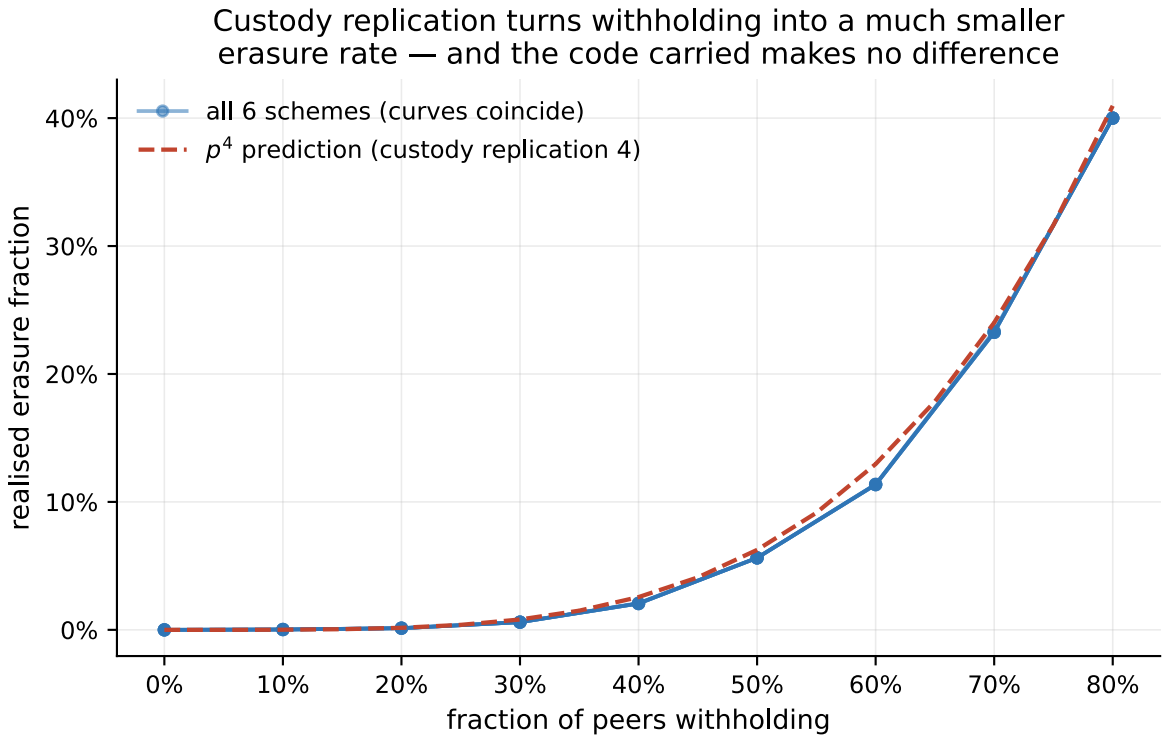
| | tolerance | must fetch | of |
|---|---|---|---|
| 1D-RS | 0.50 | 4096 | 8192 |
| 2D-RS | 0.35 | 5492 | 8448 |
| BC ω1024 λ2 | 0.33 | 5489 | 8192 |

So a weaker decoder forces a **larger download at identical storage cost**. This is the benchmark report's "surviving data needed" figure, no longer sitting in a table but actually driving a network transfer. Setting the fetch target from *k* alone instead made every scheme sit at 42% erasure even with a perfect network, and BC then "failed" at zero adversaries — an artifact of the fetch policy, not a property of the code.

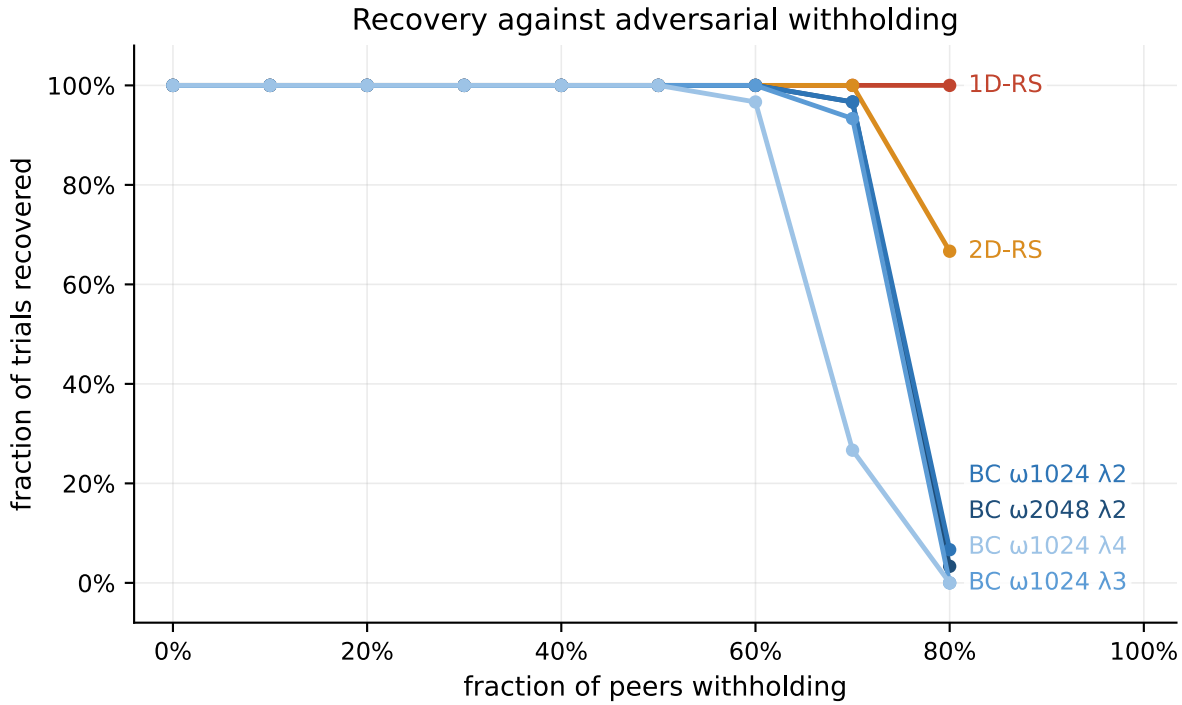
## Results

The figures below come from three sweeps, all with 30 trials per point: **A** — one blob at 50 Mbps, six schemes, withholding swept 0–80%; **B** — six blobs at 25 Mbps, the same schemes, 0–60%; **C** — peer count 8→256 at 30% and 60% withholding. Raw rows are in the accompanying JSON.

Decode times come from this project's Python/`galois` implementation and are two to three orders of magnitude slower than production field arithmetic. Absolute figures describe *this code*; comparisons between schemes are the transferable part. `--speedup N` divides decode by N to ask what production arithmetic would look like.




**Custody replication, not the code, is what absorbs peer failure.** A cell is lost only when every one of its four custodians withholds, so the erasure the code actually sees is roughly the fourth power of the fraction of bad peers. The dashed line is that p⁴ prediction, computed from the model and not fitted; the solid line is what 30 trials per point produced. Measurement sits slightly below prediction throughout, because a node that gets an early refusal still has time to retry another custodian. The practical reading: 40% of peers withholding costs about 2% of the data.




**The benchmark's tolerance ordering reappears through a completely different mechanism.** No erasure rate is injected here — losses come from peers refusing to answer. Yet the schemes fail in exactly the order the coding benchmark predicted, and at the adversarial rates that produce their measured erasure thresholds. BC ω1024 λ4 breaks first (tolerance 0.20, and 70% withholding yields 0.233 erasure); 2D-RS and the λ2 rows hold until 80% withholding drives erasure to 0.400; 1D-RS, tolerating 0.50, never fails anywhere in this sweep. 30 trials per point.

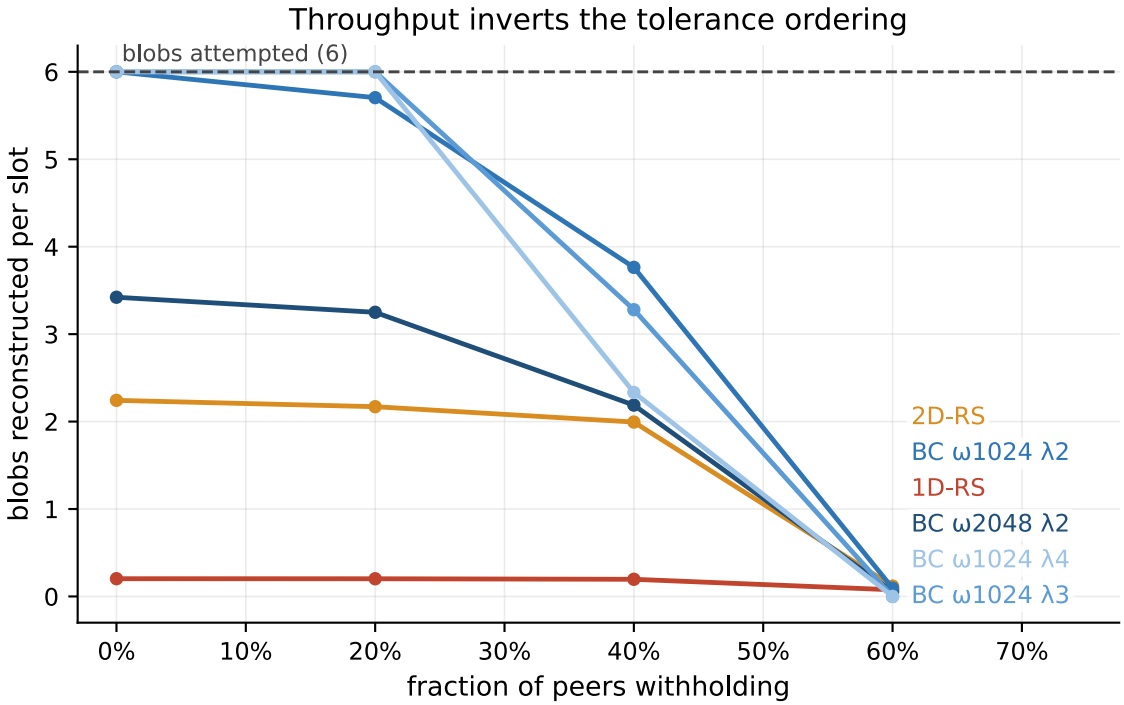
### One blob per slot, 50 Mbps (selected rates; full sweep in runA.json)

| scheme | adversarial | recovered | erasure | KiB | gather | decode | total | blobs/slot |
|---|---|---|---|---|---|---|---|---|
| 1D-RS | 0% | 30/30 | 0.000 | 262 | 0.14s | 0.00s | 0.14s | 1.00 |
| 1D-RS | 20% | 30/30 | 0.001 | 262 | 0.19s | 0.86s | 1.05s | 1.00 |
| 1D-RS | 40% | 30/30 | 0.021 | 257 | 0.43s | 5.92s | 6.34s | 1.00 |
| 1D-RS | 60% | 30/30 | 0.114 | 232 | 0.78s | 23.36s | 24.13s | 0.50 |
| 1D-RS | 70% | 30/30 | 0.233 | 201 | 1.22s | 45.19s | 46.41s | 0.26 |
| 1D-RS | 80% | 30/30 | 0.400 | 157 | 3.22s | 70.33s | 73.55s | 0.16 |
| 2D-RS | 0% | 30/30 | 0.000 | 270 | 0.17s | 0.00s | 0.17s | 1.00 |
| 2D-RS | 20% | 30/30 | 0.001 | 270 | 0.25s | 0.41s | 0.66s | 1.00 |
| 2D-RS | 40% | 30/30 | 0.021 | 265 | 0.57s | 2.46s | 3.03s | 1.00 |
| 2D-RS | 60% | 30/30 | 0.114 | 239 | 1.02s | 3.44s | 4.46s | 1.00 |
| 2D-RS | 70% | 30/30 | 0.234 | 207 | 1.72s | 5.22s | 6.94s | 1.00 |
| 2D-RS | 80% | 20/30 | 0.400 | 162 | 4.00s | 6.03s | 10.03s | 0.67 |
| BC ω2048 λ2 | 0% | 30/30 | 0.000 | 262 | 0.18s | 0.00s | 0.18s | 1.00 |
| BC ω2048 λ2 | 20% | 30/30 | 0.001 | 262 | 0.32s | 0.07s | 0.39s | 1.00 |
| BC ω2048 λ2 | 40% | 30/30 | 0.021 | 257 | 0.65s | 0.49s | 1.14s | 1.00 |
| BC ω2048 λ2 | 60% | 30/30 | 0.114 | 232 | 1.16s | 2.33s | 3.49s | 1.00 |
| BC ω2048 λ2 | 70% | 29/30 | 0.233 | 201 | 2.72s | 5.10s | 7.78s | 0.97 |
| BC ω2048 λ2 | 80% | 1/30 | 0.400 | 157 | 4.00s | 7.90s | 11.90s | 0.03 |
| BC ω1024 λ2 | 0% | 30/30 | 0.000 | 262 | 0.18s | 0.00s | 0.18s | 1.00 |
| BC ω1024 λ2 | 20% | 30/30 | 0.001 | 262 | 0.32s | 0.04s | 0.36s | 1.00 |
| BC ω1024 λ2 | 40% | 30/30 | 0.021 | 257 | 0.65s | 0.27s | 0.93s | 1.00 |
| BC ω1024 λ2 | 60% | 30/30 | 0.114 | 232 | 1.16s | 1.25s | 2.41s | 1.00 |
| BC ω1024 λ2 | 70% | 29/30 | 0.233 | 201 | 2.72s | 2.68s | 5.36s | 0.97 |
| BC ω1024 λ2 | 80% | 2/30 | 0.400 | 157 | 4.00s | 4.21s | 8.21s | 0.07 |
| BC ω1024 λ3 | 0% | 30/30 | 0.000 | 262 | 0.21s | 0.00s | 0.21s | 1.00 |
| BC ω1024 λ3 | 20% | 30/30 | 0.001 | 262 | 0.50s | 0.05s | 0.55s | 1.00 |
| BC ω1024 λ3 | 40% | 30/30 | 0.021 | 257 | 0.90s | 0.35s | 1.25s | 1.00 |
| BC ω1024 λ3 | 60% | 30/30 | 0.114 | 232 | 2.00s | 1.60s | 3.60s | 1.00 |
| BC ω1024 λ3 | 70% | 28/30 | 0.233 | 201 | 4.00s | 3.34s | 7.34s | 0.93 |
| BC ω1024 λ3 | 80% | 0/30 | 0.400 | 157 | 4.00s | — | — | 0.00 |
| BC ω1024 λ4 | 0% | 30/30 | 0.000 | 262 | 0.33s | 0.00s | 0.33s | 1.00 |
| BC ω1024 λ4 | 20% | 30/30 | 0.001 | 262 | 0.98s | 0.06s | 1.04s | 1.00 |
| BC ω1024 λ4 | 40% | 30/30 | 0.021 | 257 | 2.75s | 0.44s | 3.19s | 1.00 |

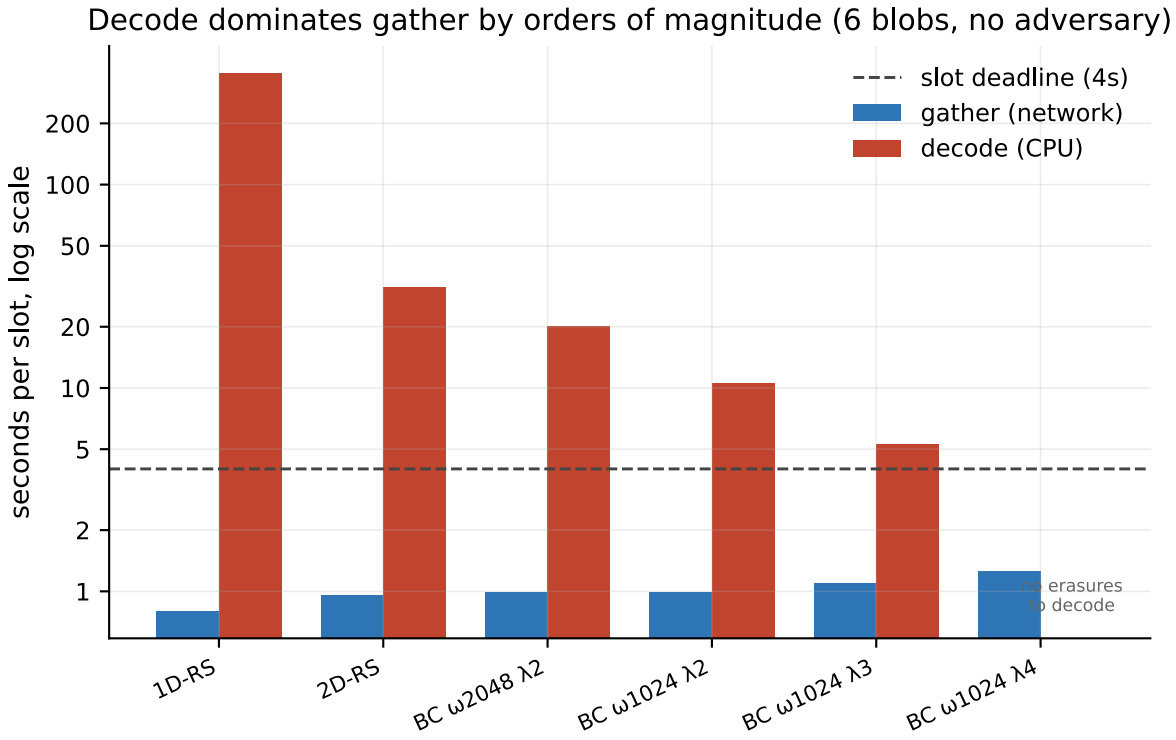
| scheme | adversarial | recovered | erasure | KiB | gather | decode | total | blobs/slot |
|---|---|---|---|---|---|---|---|---|
| BC ω1024 λ4 | 60% | 29/30 | 0.114 | 232 | 4.00s | 1.92s | 5.92s | 0.97 |
| BC ω1024 λ4 | 70% | 8/30 | 0.233 | 201 | 4.00s | 3.50s | 7.50s | 0.27 |
| BC ω1024 λ4 | 80% | 0/30 | 0.400 | 157 | 4.00s | — | — | 0.00 |

### Six blobs per slot, 25 Mbps — where bandwidth starts to bite

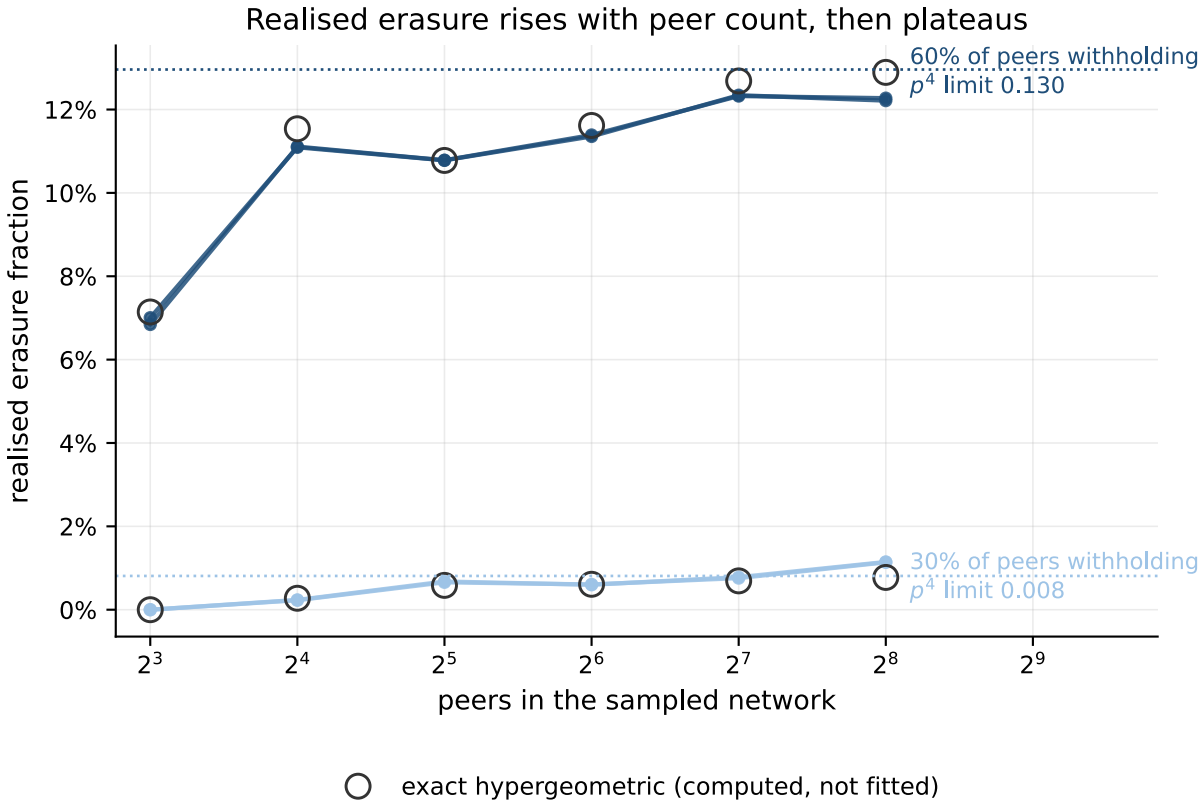
| scheme | adversarial | recovered | erasure | KiB | gather | decode | total | blobs/slot |
|---|---|---|---|---|---|---|---|---|
| 1D-RS | 0% | 30/30 | 0.320 | 1068 | 0.80s | 354.07s | 354.87s | 0.20 |
| 1D-RS | 20% | 30/30 | 0.321 | 1068 | 1.62s | 354.39s | 356.01s | 0.20 |
| 1D-RS | 40% | 30/30 | 0.333 | 1051 | 3.21s | 363.70s | 366.91s | 0.20 |
| 1D-RS | 60% | 15/30 | 0.514 | 754 | 4.00s | 470.10s | 474.10s | 0.08 |
| 2D-RS | 0% | 30/30 | 0.212 | 1277 | 0.95s | 31.15s | 32.10s | 2.24 |
| 2D-RS | 20% | 30/30 | 0.213 | 1276 | 1.98s | 31.19s | 33.18s | 2.17 |
| 2D-RS | 40% | 30/30 | 0.248 | 1222 | 3.72s | 32.38s | 36.10s | 1.99 |
| 2D-RS | 60% | 2/30 | 0.517 | 769 | 4.00s | 36.44s | 40.44s | 0.12 |
| BC ω2048 λ2 | 0% | 30/30 | 0.156 | 1326 | 0.99s | 20.05s | 21.04s | 3.42 |
| BC ω2048 λ2 | 20% | 30/30 | 0.157 | 1325 | 2.03s | 20.13s | 22.16s | 3.25 |
| BC ω2048 λ2 | 40% | 29/30 | 0.214 | 1242 | 3.79s | 28.02s | 31.80s | 2.19 |
| BC ω2048 λ2 | 60% | 1/30 | 0.511 | 756 | 4.00s | 40.87s | 44.87s | 0.05 |
| BC ω1024 λ2 | 0% | 30/30 | 0.156 | 1326 | 0.99s | 10.55s | 11.54s | 6.00 |
| BC ω1024 λ2 | 20% | 30/30 | 0.157 | 1325 | 2.03s | 10.59s | 12.62s | 5.70 |
| BC ω1024 λ2 | 40% | 29/30 | 0.214 | 1242 | 3.79s | 14.71s | 18.49s | 3.76 |
| BC ω1024 λ2 | 60% | 1/30 | 0.511 | 756 | 4.00s | 21.26s | 25.26s | 0.10 |
| BC ω1024 λ3 | 0% | 30/30 | 0.062 | 1474 | 1.10s | 5.26s | 6.35s | 6.00 |
| BC ω1024 λ3 | 20% | 30/30 | 0.063 | 1473 | 2.23s | 5.30s | 7.53s | 6.00 |
| BC ω1024 λ3 | 40% | 24/30 | 0.186 | 1292 | 3.93s | 13.65s | 17.56s | 3.28 |
| BC ω1024 λ3 | 60% | 0/30 | 0.511 | 756 | 4.00s | — | — | 0.00 |
| BC ω1024 λ4 | 0% | 30/30 | 0.000 | 1572 | 1.26s | 0.00s | 1.26s | 6.00 |
| BC ω1024 λ4 | 20% | 30/30 | 0.001 | 1570 | 2.72s | 0.38s | 3.10s | 6.00 |
| BC ω1024 λ4 | 40% | 16/30 | 0.179 | 1304 | 4.00s | 12.46s | 16.46s | 2.33 |
| BC ω1024 λ4 | 60% | 0/30 | 0.511 | 756 | 4.00s | — | — | 0.00 |




**Throughput at six blobs per slot and 25 Mbps — the ordering inverts.** 1D-RS tolerates the most erasure of any scheme here and reconstructs the fewest blobs, roughly 0.2 of the six, because one 8192-length polynomial division costs roughly 59s per blob. BC ω1024 λ2, λ3 and λ4 keep up with all six until 20% withholding. Note this is throughput *implied by* the measured time — blobs attempted, scaled by the measured recovery rate and by whether the total fits the 12s slot — not an independently measured rate.




**Where the time goes, six blobs, no adversaries, log scale.** Gather and decode are drawn as separate bars rather than stacked, because stacking on a log axis would misrepresent the sum. Every scheme gathers in about one second; decode spans three orders of magnitude above that. This is the central result of the network layer: at every setting tried, arithmetic rather than bandwidth or latency decides whether a scheme fits the slot. BC ω1024 λ4 shows no decode bar because it has the weakest decoder and therefore fetches the most (80% of the codeword); with nothing missing there is nothing to reconstruct. It buys its speed with bandwidth.




**Growing the network makes things slightly worse, up to a limit.** At 60% withholding and fixed replication 4, custodians are drawn without replacement, so in a small network it is combinatorially hard for all four custodians of a cell to be adversarial — with 8 peers only 5 are bad and there are few ways to choose 4 of them. As the network grows this constraint relaxes and the loss rate climbs toward the independent-sampling limit $p^4 = 0.130$. The open markers are the exact hypergeometric probability, again computed rather than fitted; both schemes sit on it. The lesson for scalability claims is that what protects a cell is the *fraction* of the network custodying it, so peer count cannot be varied without saying what replication does too.

## What the results show

**Decode dominates; the network barely registers.** Gathering takes 0.14–3.0 seconds. Decoding takes up to 466. At every parameter setting tried, the bottleneck is arithmetic, not the link or the latency. Cutting bandwidth fivefold, from 25 Mbps to 5 Mbps, moved 2D-RS's total from 32.1s to 33.1s — a 3% change, because the extra second of transfer disappeared next to 31 seconds of decode. At six blobs the spread is starker still: every scheme gathers in about one second, while decode ranges from 0.0s to 354.1s.

That single finding reframes the comparison. The coding benchmark reported decode cost as one column among several; placed in a slot budget it becomes the only column that decides anything.

**1D-RS tolerates the most and delivers the least.** At 80% of peers withholding — a realised erasure fraction of 0.400 — it is the only scheme still recovering every trial, 30/30, where 2D-RS manages 20/30 and the block-circulant rows collapse to 2/30 or worse. It needed 73.5 seconds to do it, six times the slot. At six blobs it reconstructs 0.20 of them per slot. Best fault tolerance in the study, worst throughput, and the two facts have the same cause: one enormous polynomial division.

**Three block-circulant rows keep up with six blobs; nothing else does.** The ω1024 rows at λ2, λ3 and λ4 all reconstruct all six, against BC ω2048 λ2's 3.42, 2D-RS's 2.24 and 1D-RS's 0.20. Their decoders are the weakest in the study — λ2 tolerates 0.33 where 1D-RS tolerates 0.50 — but λ2 decodes roughly 33× faster (10.6s against 354.1s for the same six blobs), and inside a deadline that trade is strongly favourable.

There is a visible progression inside those three rows, and it is a bandwidth-for-compute trade. λ2 fetches 1326 KiB and decodes for 10.6s; λ3 fetches 1474 KiB and decodes for 5.3s; λ4 fetches 1572 KiB and decodes for 0.0s, because its tolerance is so low that it must fetch essentially the whole codeword and then has nothing left to reconstruct. The weakest decoder ends up doing no decoding at all — by paying for it on the wire.

**Network-level custody replication absorbs most peer failure before the code is involved.** At 70% adversarial peers the realised erasure fraction is only 0.237, because a cell dies only when all four of its custodians do. The erasure code's tolerance is not the binding constraint until the adversarial fraction is very high — which suggests custody replication and code strength are substitutes, and that the cheaper one should be bought first.

**More peers is not automatically safer, and can be worse.** Sweeping peer count at fixed custody replication produced a result worth stating because it runs against intuition. Custodians are drawn without replacement, so in a small network it is combinatorially hard for all four custodians of a cell to be adversarial: with 8 peers and 60% withholding only 5 peers are bad, and there are just 5 ways to choose 4 of them out of 70. As the network grows that constraint relaxes and the loss rate climbs toward the independent-sampling limit $p^4$. Measured erasure at 60% withholding runs 0.068 at 8 peers, 0.111 at 16, 0.114 at 64 and 0.123 at 256, against the exact hypergeometric values 0.071, 0.115, 0.116 and 0.129 — agreement within 0.006 at every point. At 30% withholding 2D-RS took 0.33s with 8 peers and 1.73s with 64, entirely because the larger network lost more cells and therefore had more to decode.

The lesson is that what protects a cell is not the peer count but the *fraction of the network custodying it*. Growing the network while holding replication fixed quietly dilutes that fraction. Any scalability claim about node count has to say what happens to replication at the same time, or it is not saying anything.

**A weaker decoder costs bandwidth as well as tolerance.** BC fetched 1326 KiB where 1D-RS fetched 1068 KiB for the same six blobs, because it needs a larger surviving fraction to decode at all. At equal storage, the scheme with the weaker decoder pays on the wire too — a cost the isolated coding benchmark could not show.

## Validation, and two bugs it caught

The calibrated decode-cost model was checked against `--real-decode`, which hands the emergent clustered mask to the actual decoders:

| point | calibrated | real |
|---|---|---|
| 2D-RS @ 40% adversarial | 3.18s | 2.28s |
| 2D-RS @ 60% | 4.80s | 2.65s |
| BC ω1024 λ2 @ 40% | 0.72s | 0.58s |
| BC ω1024 λ2 @ 60% | 2.21s | 2.02s |

Same order of magnitude, real slightly faster, and every trial recovered — so clustered erasures at these levels decode as cleanly as i.i.d. ones.

Two errors surfaced this way and are fixed:

1. **Zero-erasure rounds invented work.** The cost curve was held flat at its lowest *measured* rate (0.02), so a round with nothing missing was charged several seconds of decoding. `--real-decode` reported

0.00s and exposed it.

2. **Gather time was reporting the deadline back.** The first version ran the environment to the deadline and reported `env.now`, which is the deadline by construction. It now records the moment the node actually held enough cells to start decoding.

## Limitations

**One topology.** The brief asks for "standard network architectures", plural. This models a flat network with uniform random custody. Structured overlays, clustered or geographically correlated latency, and churn are not represented. This is the largest remaining gap.

**One-hop fetches only.** A node requests cells directly from custodians. There is no gossip propagation, no forwarding, no upload-side cost — so this measures a node's *download* experience, not network-wide load.

**Decode cost is Python-speed.** Stated above and worth restating: absolute timings are two to three orders of magnitude off production, which is what `--speedup` exists for. Relative comparisons hold.

**Statistical treatment is lighter than the coding benchmark's.** 8–10 trials per point with no confidence intervals, against the benchmark's Wilson intervals and per-scheme trial budgets. Adequate for the size of the effects being reported here — a 30× throughput gap does not need a confidence interval — but it should be tightened before any close call is claimed.

## Running it

```
pip install simpy galois numpy matplotlib

python3 network_sim.py                                  # defaults
python3 network_sim.py --blobs 6 --downlink 25          # bandwidth pressure
python3 network_sim.py --peers 8,16,64,256              # sweep network size
python3 network_sim.py --real-decode --trials 3         # ground truth
python3 network_sim.py --speedup 100                    # production arithmetic
python3 network_sim.py --figures figs.md                # SVG, docs-styled
python3 network_sim.py --json results.json              # machine-readable
python3 network_sim.py --no-visuals                     # table only
```

`--peers` and `--adversarial` both accept comma-separated lists and are swept as a cross product with the scheme list, so a single invocation can cover scheme x network size x adversarial pressure.

### What it draws

Every run prints three terminal views beneath the results table, because the table alone buries the one thing that matters:

- **Throughput** — a bar per configuration, blobs reconstructed per slot.
- **Where the time goes** — gather and decode as one stacked bar per configuration, with a marker showing where the 12-second slot falls. This is the figure that makes the central finding obvious at a glance: the network segment is a sliver and the arithmetic segment is everything. A segment that rounds to zero width is still drawn one cell wide, so "too small to see" stays distinguishable from "absent".

- **Recovery grid** — scheme x peer count against adversarial fraction, marked ● all trials recovered, ◐ some, ○ none.

`--figures out.md` writes the first two as inline SVG in the same style as the rest of `docs/`, so they can be pasted into the write-up directly.

The figures in *this* report are not those, and are not drawn by hand either. They are generated from the committed run data by `make_figures.py`:

```
python3 make_figures.py --only netA netB netC
```

`docs/runA.json` holds the adversarial sweep at one blob, `runB.json` the same sweep at six, and `runC.json` the peer-count sweep. Regenerating from those files rather than redrawing means a figure cannot drift out of step with the numbers quoted beside it, and the hypergeometric and $p^4$ curves are computed in the plotting code rather than transcribed.

Scheme names match `benchmark_results.json`, so every parameterisation the coding benchmark measured can be driven through the network layer without touching either file.